

Assignment — E-Commerce Product Store

Monthly Performance Evaluation · HTML + CSS + JavaScript

MERN Stack + AI Engineering Bootcamp

Total Marks: 100 · Weightage: 60% Assignment + 40% Interview

Submission: GitHub Repository + Live Demo + Video Walkthrough

Overview

You will build a fully functional e-commerce product store using **HTML, CSS, and vanilla JavaScript only**. Product data will be fetched from a real, free REST API. This assignment tests everything covered so far — semantic HTML, responsive CSS, DOM manipulation, fetch/async-await, array methods, closures, localStorage, and event handling.

API to use:

```
https://fakestoreapi.com/products
// Free, no API key needed. Returns 20 real products with
// images, categories, ratings, prices, and descriptions.
```

Restrictions — Read Carefully

- Only **HTML, CSS, and vanilla JavaScript** — no frameworks of any kind
- No CSS frameworks — no Bootstrap, no Tailwind, no Materialize
- No JavaScript libraries — no jQuery, no Lodash, no Axios
- You **MAY** use Google Fonts and Font Awesome icons
- All JavaScript must be in separate **.js files** — no inline `<script>` tags in HTML
- All CSS must be in separate **.css files** — no inline style attributes in HTML
- No use of **alert()** or **confirm()** anywhere — build your own UI for all messages

Section A — Product Listing (Core)

- 1 Fetch all products from the API using **fetch()** with **async/await** on page load.
- 2 While products are loading, display a **skeleton loading screen** — placeholder cards with a CSS pulse/shimmer animation. These must look like real product cards, not just grey boxes.

- 3 If the API call **fails**, show a clearly styled error state with a **'Try Again'** button that retries the fetch when clicked.
- 4 Display products in a **responsive grid** — 3 columns on desktop, 2 on tablet, 1 on mobile.
- 5 Each product card must display: product image (clicking anywhere on the card opens the product modal), category badge (styled pill/tag), product title truncated to 2 lines using CSS (-webkit-line-clamp), star rating display using actual star icons (not just a number), price, and an **'Add to Cart'** button.
- 6 Show a **product count** at the top of the page — e.g. 'Showing 20 products' — that updates dynamically when filters are applied.

Section B — Filter, Search & Sort (Must All Work Simultaneously)

- 7 **Search bar** — user types a product name, results filter in real time, case-insensitive.
- 8 **Category filter buttons** — generate category buttons **dynamically from the API data** (do NOT hardcode any category names in HTML). Clicking a category shows only that category's products. An 'All' button resets the filter.
- 9 **Sort dropdown** with 4 options: Price Low to High, Price High to Low, Rating Best First, Name A to Z.
- 10 All three — search, category filter, and sort — must **work together simultaneously**. Example: if a user is viewing 'electronics', has typed 'phone' in search, and selected 'Price Low to High', all three must apply at the same time without any breaking each other.
- 11 A **'Clear All Filters'** button that resets search input, active category, and sort order back to default in a single click.
- 12 If no products match the current filter combination, show a styled **'No products found'** empty state with a helpful message.

Section C — Product Detail Modal

- 13 Clicking a product card (anywhere except the 'Add to Cart' button) opens a **modal overlay**.
- 14 The modal must show: full-size product image, full product title (not truncated), full product description, category badge, price, star rating + number of reviews (e.g. 4.2 stars · 120 reviews), a **quantity selector** (+ / -) starting at 1 minimum, and an 'Add to Cart' button that adds the chosen quantity.
- 15 Modal closes when clicking the **X button**, clicking **outside** the modal overlay, or pressing the **Escape key**.

Section D — Shopping Cart

- 16 A **cart icon in the header** with an item count badge (e.g. 3 items). The badge is hidden when the cart is empty.

- 17 Clicking the cart icon opens a **cart sidebar/drawer** that slides in from the right side of the screen.
- 18 Each cart item must show: small product image, product title (truncated to 1 line), unit price, **quantity control buttons (+ and -)**. The minus button at quantity 1 does NOT delete the item — it stays at 1. A separate **Remove (x) button** deletes the item from cart entirely.
- 19 Below all cart items: a **subtotal** that updates dynamically every time quantity changes or an item is removed.
- 20 If the cart is empty, show an **empty cart message** and a 'Continue Shopping' button that closes the drawer.
- 21 A **Checkout button** at the bottom — clicking it shows an **order confirmation modal** listing all ordered items, quantities, and the total. The cart clears after checkout.

Section E — LocalStorage Persistence

- 22 Cart contents must **survive a page refresh**. When the page reloads, the cart is restored exactly as the user left it — same items, same quantities.
- 23 The **cart count badge** must reflect the correct count immediately on page load, before any user interaction.
- 24 Dark/light mode preference (see Section F) must also be saved to localStorage and restored on load.

Section F — Dark / Light Mode

- 25 A **toggle button in the header** switches between dark mode and light mode.
- 26 The active mode is saved to localStorage and restored on page load — **no flash of the wrong theme** when the page first opens.
- 27 The toggle button icon must change to reflect the current mode (e.g. moon icon for dark mode, sun icon for light mode).

Section G — Responsive Design

- 28 Fully responsive at all screen sizes: mobile (320px minimum), tablet (768px), desktop (1200px+).
- 29 On mobile, the header must have a **hamburger menu** that opens and closes navigation links.
- 30 Cart drawer takes **full screen width** on mobile screens.

3
1 Product grid adjusts: 3 columns desktop, 2 columns tablet, 1 column mobile.

3
2 All modals are scrollable on small screens and do not overflow or clip any content.

Code Quality Requirements

- Use **const** and **let** throughout — never var
- All data manipulation must use array methods: `.filter()`, `.map()`, `.sort()`, `.find()` — no manual for loops for data processing
- Functions must be small and focused — no single function longer than 20 lines
- Use **meaningful variable and function names** — no single-letter names except loop counters
- No repeated/duplicate code — extract shared logic into reusable functions
- Use **template literals** (backtick strings) for all HTML generation — no string concatenation with +
- Handle all edge cases: API failure, empty cart, no search results, image load failure (show a placeholder image if the product image fails to load)
- Add comments explaining any non-obvious logic

Required File Structure

```
ecommerce-store/  
  ■■■ index.html  
  ■■■ css/  
    ■■■ style.css      (main styles)  
    ■■■ dark-mode.css  (dark mode overrides)  
    ■■■ skeleton.css   (skeleton loading animation)  
  ■■■ js/  
    ■■■ app.js         (main entry point)  
    ■■■ api.js         (all fetch logic)  
    ■■■ products.js    (product rendering logic)  
    ■■■ cart.js        (all cart logic)  
    ■■■ filters.js     (search/filter/sort logic)  
    ■■■ ui.js          (modal, drawer, dark mode)  
  ■■■ README.md
```

Bonus Tasks (Extra Marks — Max 5)

- ★ **Wishlist feature** — heart icon on product cards saves items to a wishlist stored in localStorage. A separate 'Wishlist' view or section shows saved items.
- ★ **Pagination or Load More** — instead of showing all 20 at once, show 8 products with a 'Load More' button that reveals the next batch.
- ★ **Product comparison** — select 2 or 3 products and compare their specs side by side in a modal.
- ★ **Debounced search (using closures)** — search input waits 300ms after the user stops typing before filtering. Write your own debounce function using a closure — do NOT use any library.
- ★ **Smooth animations** — products animate in on load (fade + slide up), modal has entrance/exit transitions, cart drawer slides smoothly.

README Requirements (Mandatory)

- 1 **Project title and brief description** — 2 to 3 sentences

2	Live demo link — deploy to GitHub Pages, Netlify, or Vercel (all free). A project with no live link loses 5 marks.
3	Features list — bullet points of everything you implemented
4	Screenshots — at least 3: desktop product grid, mobile view, cart drawer open
5	Technologies used
6	How to run locally — step by step instructions
7	What I learned — an honest paragraph about challenges you faced and what you learned from them
8	Video walkthrough — a screen recording of 3 to 5 minutes maximum where you walk through all features live. Upload to YouTube (unlisted is fine) or Google Drive and paste the link. This is mandatory — submissions without a video lose 10 marks.

Submission Instructions

Create a GitHub repository named exactly: **ecommerce-store-[your-name]** (e.g. ecommerce-store-asad-khan)

- All code committed and pushed before the deadline
- Repository must be **public** — private repos will not be graded
- Paste your **GitHub repo link + live demo link** in the submission form or DM
- Late submissions lose **10 marks per day**

Marks Breakdown

Category	Marks
Product listing — fetch, skeleton loading, error state + retry	10
Filter + Search + Sort — all three working simultaneously	15
Product detail modal — full info, quantity, keyboard close	10
Shopping cart — add, remove, quantity control, subtotal, checkout	15
LocalStorage persistence — cart and dark mode survive reload	10
Dark / Light mode — toggle, persistence, no flash on load	5
Responsive design — grid, hamburger menu, mobile cart drawer	10
Code quality — const/let, array methods, file structure, comments	10
README + screenshots + live demo + video walkthrough	10
Bonus tasks (each worth 2-3 marks, maximum 5 marks total)	5
TOTAL	100

Note on Grading: The interview (40% weightage) will be based on your own submitted project. You will be asked to explain your code — how your filter + sort + search works together, how you structure your cart array, how localStorage is read and restored, and what happens when the API fails. Build it yourself so you can explain every line.